

AWS Certified Generative AI Developer — Professional

The Use Case

- Global financial institution — 50 million customers, 30 countries
- Requirements:** consistent handling across regions and languages, strict regulatory and privacy compliance, **audit trails of all AI interactions**, personalised guidance from customer history, seamless escalation to human agents with full context

Where Task 1.6 Sits

Task	Focus
1.1	Build the system
1.2	Select and manage the model
1.3	Prepare data before it reaches the model
1.4	Build and maintain the vector store
1.5	Retrieval quality
1.6	How you talk to the model — and how that is governed

The organising idea: a prompt is a code artifact. In a regulated industry, changing a prompt is like deploying to production — it needs review, approval, versioning and an audit trail. One altered line could have the AI giving wrong financial advice to 50 million customers.

1. Model Instruction Frameworks

Three layers of control, each doing something the others cannot.

Prompt Management — role definition

```
PERSONA:      "You are a banking assistant"
CONSTRAINTS:  "You may NOT give investment advice"
BEHAVIOUR:    "Always include the fee disclaimer"
ESCALATION:   "Route complaints to a human agent"
```

The system prompt persists across the whole conversation. In banking, the **limitations** matter most — the AI must know what it is *not* allowed to do.

Bedrock Guardrails — content filtering with severity

- Four content filter categories:** hate speech, insults, sexual content, violence
- Each configurable at LOW / MEDIUM / HIGH severity.** This is a threshold: HIGH blocks only the most severe content; LOW blocks even mild content, so it is the stricter setting.
- Banking configuration:** high severity thresholds on regulated activities — unauthorised financial advice, speculation on market movements. These are regulatory violations, not just bad answers.
- Guardrails also cover **denied topics**, **word filters**, **PII filters**, and **contextual grounding** (the hallucination check from Task 1.1)

Guardrails sit outside the model and outside the prompt. One guardrail applies across multiple foundation models, and it cannot be bypassed by changing the prompt or by prompt injection.

JSON Schema response templates

```
{
  "account_number": "string (masked)",
  "balance":       "number",
  "currency":      "string",
  "fees":          [{"name": "string", "amount": "number"}],
  "disclaimer":    "string (REQUIRED)"
}
```

- Downstream processing** becomes reliable — parsing is guaranteed
- Compliance:** making `disclaimer` a required field means the model cannot omit it. Putting compliance into the schema removes the option to forget it.

How the three combine

Prompt Management

-> "behave like this"

(instruct)

Guardrails

-> "never say this"

(block)

JSON Schema

-> "answer in this shape"

(structure)

Transcript result: 97% reduction in compliance violations from these three layers together.

2. Interactive AI Systems

Step Functions — conversation orchestration and clarification loops

"My transfer didn't go through" — which transfer? Wire, ACH, international? Guessing produces a wrong answer.

```
Customer message
-> detect intent
-> confidence low?  --yes--> ask a clarifying question
    | no
    |
handle it              answer returns -> re-detect
```

- Step Functions holds the **conversation state** — which stage, what has been asked, how many times
- **Lambda alone can't do this** — it's stateless; multi-turn dialogue needs a state machine
- Also handles **escalation to a human agent with full context**

Amazon Comprehend — custom intent classification

- Generic Comprehend doesn't know banking, so a **custom classifier** is trained on banking terminology
- **Transcript: 200+ distinct customer intents** for precise routing
- **Why not ask the model?** Comprehend is **cheaper, faster and deterministic** — the same input always gives the same intent, which matters in banking. It also returns a confidence score, which drives the clarification loop.

DynamoDB — conversation history and session management

```
PK: session_id
SK: timestamp
    role, content, intent, customer_id
```

- Single-digit millisecond latency — read and written on every turn
- **TTL set to 90 days** per regulatory data-retention requirements

TTL is a compliance control, not housekeeping. DynamoDB expires items automatically, so the retention policy is enforced by the system — no cleanup script to write, no chance of forgetting.

The **session ID** ties together history, customer profile, detected intent and workflow state. When a case escalates, the human agent receives the full context. **Transcript: 78% reduction in customers having to repeat information.**

As in Task 1.3: **the model remembers nothing** — memory is something you build.

3. Prompt Management and Governance

Parameterized templates

```
You are the banking assistant for {{bank_name}}.
Customer: {{customer_name}}, tier: {{customer_tier}}
Account: {{account_type}}
Region: {{region}} – {{regulatory_framework}} applies

Required: {{required_disclaimers}}

Question: {{query}}
```

One template serves 30 countries. Change region and the correct regulatory framework and disclaimers follow. Without this you maintain 30 separate prompts, 29 of which drift out of date.

Three-tier approval workflow

```
1. Banking expert      -> is the content correct?
2. Compliance reviewer -> does it meet regulation?
3. System administrator -> deploy
```

- **Distinct roles:** author, reviewer, administrator
- Three separate people because an author cannot see their own errors, and compliance is a different skill from banking product knowledge
- Same deliberate human gate as the **clinical validation step in Task 1.2** — automating the pipeline does not mean removing the human

Versioning

v7 live, v8 in approval; if v8 misbehaves, roll back to v7 in seconds. **No code deployment involved** — that is the point of Prompt Management.

CloudTrail vs CloudWatch Logs

Service	Records
CloudTrail	Every Bedrock API call — which prompt (and version), by whom, when, which model. Metadata.
CloudWatch Logs	Model invocation detail — the full prompt sent and response returned , token counts, latency. Content.

- **Transcript: CloudTrail logs retained for seven years** to meet financial regulation
- Why: three years later a customer complains about advice they received, and the regulator asks which prompt was live at the time
- **Both are needed** — CloudTrail says who did it, CloudWatch says what happened
- S3 can hold the template repository for version history, backup and CI/CD integration

4. Quality Assurance Systems

Traditional testing doesn't transfer: the model is non-deterministic, and "correct" isn't an exact string.

Output validation — Lambda, in real time

required disclaimers present? fee information accurate? escalation trigger correct? JSON schema satisfied? any unauthorised advice?

Checks for	
Guardrails	Harmful content
Output validation	Business rules — disclaimers, fee accuracy, escalation logic

Guardrails has no way to know your bank requires a disclaimer with every fee. That is custom logic.

Edge case testing — Step Functions

Normal cases pass. Failures live at the edges.

- **Transcript's examples: joint accounts, international transfers**
- Others: closed accounts, negative balances, multi-currency, minor's accounts, deceased account holders
- Step Functions runs the suite systematically — hundreds of cases, in parallel, results collected. Run before every prompt version ships.

CloudWatch KPI monitoring

KPI	What it reveals
Accuracy rate	Are answers correct?
Latency	Response speed
Escalation rate	The subtlest signal — a sudden rise means the AI can't cope, while error rate stays normal. Nothing else surfaces this.
Customer satisfaction	Perceived quality
Error rate	Outright failures

Regression testing — the critical one

You write v8 to fix joint accounts. It works — **and quietly breaks international transfers**. Changing a prompt in one place affects behaviour elsewhere.

Test case	v7	v8	
Joint account	62%	-> 94%	improved
Intl transfer	91%	-> 73%	REGRESSED
Balance inquiry	98%	-> 97%	fine

Run both versions against the same test set and compare. Same principle as the golden dataset in Task 1.2, applied to prompt versions.

Transcript: 34 potential compliance issues identified and prevented in the first three months — caught before reaching customers.

When each runs

Mechanism	Timing
Output validation	Every request, real time
Edge case testing	Before deployment
CloudWatch KPIs	Continuously in production
Regression testing	Every new prompt version

5. Enhancing FM Performance

Same model, better prompt, better answers. That is the whole point of this section.

Structured input components

Instead of making the model parse a rambling query, hand it defined fields:

```
{
  "intent":          "EARLY_WITHDRAWAL_FEE",
  "product":         "fixed_deposit",
  "amount":          500000,
  "opened_date":     "2024-03-15",
  "tenure_remaining_months": 12,
  "customer_tier":   "premium",
  "region":          "IN"
}
```

The model's capacity then goes into answering rather than interpreting. The entities extracted by Comprehend in Task 1.3 become these fields.

Output format specifications

Beyond compliance, format improves **accuracy**: told that `fee_amount` must be a number, the model calculates carefully rather than hedging with "there may be a charge". JSON or XML both work — some models respond better to XML tags.

Chain-of-Thought (CoT)

```
Think step by step:
1. Identify principal and rate
2. Calculate interest accrued to date
3. Look up the early withdrawal penalty rate
4. Apply the penalty
5. Check premium tier adjustments
6. Compute the final amount
```

Show each step, then give the answer.

Why it works: the model generates one token at a time. Writing the reasoning first puts that reasoning into its own context, so each step builds on the last. Demanding an immediate answer forces everything into a single leap — far more error-prone.

- **Audit benefit:** the reasoning is visible, so a wrong answer can be traced to the step that failed
- **Practical note:** the reasoning need not be shown to the customer — let the model reason, then extract only the final answer in post-processing
- Transcript uses CoT for complex financial calculations

Feedback loops

- **Customer satisfaction ratings** — thumbs up/down, star ratings
- **Agent corrections** — the richer signal. When an agent edits the AI's answer, that shows both *that* it was wrong and *what* the right answer was.

```
feedback accumulates
-> pattern emerges ("mortgage queries rate poorly")
-> refine that scenario's prompt
-> new version -> regression test
-> deploy -> measure again
```

Transcript: 23% improvement in customer satisfaction from these enhancements.

6. Complex Prompt Systems — Bedrock Prompt Flows

A mortgage prequalification isn't one prompt: extract the financial profile, check eligibility, calculate affordability, match products, assemble a compliant response. Each step's output feeds the next.

The flow graph

```
Input
-> [Pre-processing]  normalise terminology
-> [Prompt: profile] extract financial profile
-> [Condition]       credit score check
-> [Prompt: products] match products
-> [Post-processing] disclaimers + regional formatting
-> Output
```

Node types: Prompt, Condition, Lambda, Knowledge Base, S3, Iterator.

Conditional branching

Credit score	
750+	-> premium products flow
650-749	-> standard flow
<650	-> improvement guidance flow
Model confidence	
< 0.7	-> human agent
>= 0.7	-> automated response

Transcript routes customers down different prompting paths by financial profile and needs. **Confidence-based branching matters in banking — the AI must know when it doesn't know.**

Reusable prompt components

- The explanation of "APR" is needed in mortgage, credit card and personal loan flows. Written three times, it drifts three ways.
- One component, shared across flows, keeps the language consistent
- **Compliance payoff:** when the regulator requires new APR wording, you change it in one place

Pre-processing and post-processing

Stage	Does
Pre-processing	Normalise financial terminology ("FD", "fixed deposit", "term deposit" → one form); break input into structured fields
Post-processing	Add regulatory disclaimers , apply regional formatting (currency, dates, numbers), mask PII, validate against schema

Adding disclaimers in **post-processing** rather than relying on the prompt is the smart move — the system attaches them every time, instead of trusting the model to remember.

Flows vs Agents — final comparison

	Who decides the sequence	In banking
Flows	You — graph defined in advance	Auditable, identical every run
Agents	The model — picks its own tools	Non-deterministic, hard to audit

Regulated industries choose Flows. When the regulator asks how a decision was reached, a Flow has an answer.

Transcript: 82% of mortgage prequalification inquiries handled without human intervention.

Reported Outcomes

- **97%** reduction in compliance violations (instruction framework)
- **78%** reduction in customers repeating information (session management)
- **34** compliance issues prevented in three months (QA system)
- **23%** improvement in satisfaction (performance enhancements)
- **82%** of mortgage prequalifications fully automated (Prompt Flows)
- **67%** lower average handle time · **89%** first contact resolution · **98.7%** compliance accuracy in audits · **35%** higher CSAT · **\$43M** annual savings

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.